

Documentación del Proyecto – ZenQuotes Android (Java + LiveData)

//tuve un problema, yo trabaje con gemini y mi compañero con copilot, pero no pude activar el modo agente en gemini por lo tanto no pude

tomar las capturas, solo mi compañero pudo realizar el programa con copilot, asi que, almenos para demostrar que trabajamos, haré mi documentacion correspondiente

a mi trabajo.//

1. Resumen

Aplicación Android en Java que muestra citas motivacionales obtenidas de la API pública ZenQuotes (/api/random).

La app fue diseñada para cumplir con el esquema de la clase: usar agentes de IA (Copilot y Gemini) mediante prompts detallados, que generan código base y documentación.

Aunque no se pudo acceder al “modo agente” de Gemini en nuestra cuenta, se trabajó con los prompts diseñados para ese escenario.

2. Arquitectura y decisiones

Lenguaje: Java.

Capas:

data: modelo Quote, interfaz Retrofit, repositorio con reintentos simples.

ui: MainActivity y QuoteViewModel con LiveData.

Gestión de estados: clase QuoteUiState con loading, text, author, error.

Networking: Retrofit + Gson + OkHttp (timeouts configurados).

Control de UX: se bloquea el botón “Nueva frase” mientras está loading.

3. Modelo y contratado de API

```
public class Quote {  
    private String q;  
    private String a;  
    public String getQ() { return q; }  
    public String getA() { return a; }  
}
```

Endpoint:

GET <https://zenquotes.io/api/random> → List<Quote> (un solo objeto).

4. Prompts y trabajo con agentes

4.1 Copilot (Mi compañero, Abner Mateo)

Se usó un prompt estructurado en Java, solicitando ViewModel + LiveData + Retrofit, estados de UI y reintentos.

Copilot generó la estructura inicial (paquetes, modelo, Activity, ViewModel).

Iteraciones: ajustar dependencias, validar lista no vacía, mejorar el manejo de estados en la UI.

4.2 Gemini (Yo, Hector Mellado)

El prompt diseñado fue específico para Gemini en modo agente, pidiendo arquitectura en Java con LiveData, Retrofit, reintentos, formateo y vibecoding.

Limitación: nuestra cuenta no tiene habilitado el “modo agente” en Gemini, por lo que no se pudieron generar capturas ni código directamente desde esa modalidad.

Defensa: se siguió el flujo propuesto en clase (prompt detallado → agente responde → ajustes/iteraciones). Aunque Gemini no ejecutó en modo agente, el diseño del prompt y la documentación de su propósito muestran cómo se habría utilizado el asistente.

5. Problemas encontrados y soluciones

Falta de Agent Mode en Gemini: documentado como limitación externa, pero se justificó el proceso.

Timeouts de red: configurados en OkHttpClient.

Array vacío en la respuesta: validación antes de acceder al primer elemento.

Múltiples taps: botón deshabilitado mientras loading=true.

6. Vibecoding

Botón (variantes):

Nueva frase

Otra inspiración

Dame motivación

Refrescar cita

Más ánimo

Mensajes de share (≤ 120 caracteres):

Hoy me quedo con esta idea: «{frase}» —{autor}

Pequeño impulso del día: «{frase}» —{autor}

Para no rendirse: «{frase}» —{autor}

Recordatorio: «{frase}» —{autor}

Minidosis de ánimo: «{frase}» —{autor}

7. Aprendizaje, errores y aciertos.

-Qué funcionó bien:

Estructura de capas clara: separar data y ui permitió localizar rápido dónde modificar.

Uso de Retrofit + OkHttp con timeouts: simplificó la conexión a la API y evitó bloqueos largos.

Manejo de estados de UI (loading, data, error): facilitó mostrar mensajes claros en pantalla.

Prompts detallados: con Copilot se generó código base útil sin tener que escribir desde cero.

-Qué salió mal

Dependencias incorrectas al inicio: la primera vez que se intentó compilar, algunas versiones de librerías no coincidían (Retrofit y lifecycle). Hubo que ajustar el build.gradle.

Parsing inesperado: en una prueba inicial, la API devolvió un objeto vacío y la app mostró un `IndexOutOfBoundsException` al hacer `list.get(0)`. Se corrigió validando que la lista no esté vacía.

UI congelada al refrescar: en una de las primeras versiones, la lógica de red estaba en el hilo principal, lo que hacía que el botón pareciera colgado. Se migró al repositorio usando callbacks de Retrofit en segundo plano.

Problemas con Gemini: al no tener habilitado el “modo agente”, no se pudo obtener código directamente, lo cual retrasó la comparación entre agentes y obligó a documentar la limitación como parte del trabajo.

-Qué se podría mejorar

Incluir pruebas unitarias con `MockWebServer` para el repositorio.

Implementar dark mode y estilos Material 3 para mejor UX.

Añadir opción de compartir frases con `ACTION_SEND`.

Evaluar el uso de Inyección de Dependencias (Hilt/Dagger) para proyectos más grandes.

8. Créditos

API: ZenQuotes.

Mi Compañero: trabajo con Copilot (Java).

Yo: diseño y documentación de prompt para Gemini. (originalmente también trabajaría el programa con Gemini, pero surgió el problema)